# LIGO for TypeScript developers on Layer-1

Christian Rinderknecht
Christian.Rinderknecht@trili.tech
TriliTech

17 April 2025

Show and Tell

# A short history of LIGO

- I did my PhD at INRIA under the supervision of Michel Mauny.
- In July 2018, I joined Nomadic, where I started designing a smart contract language, similar to Pascal, that would be compiled to Michelson.
- My then-colleague Gabriel Alfour also wanted to make programming smart contracts on Tezos easier, but with a bottom-up approach: building a set of macros that expand into Michelson.
- In 2019, we joined our efforts by meeting half-way to produce what became LIGO: I contributed the language design and the front-end; he crafted the type-checker and code generator — 50%-50% in terms of lines of code (LOC).

- The next idea for LIGO was to offer different concrete syntaxes inspired from mainstream programming languages.
- Those LIGO languages were meant to be domain-specific, but not overly so: supporting special literals (natural numbers, mutez, for examples), but taking most Tezos-specific types and functions from a library.
- The front-end had to lower their input to the common pipeline, while sharing as much code as possible in the processing of the different syntaxes.
- The first syntax was PascaLIGO, as I thought it would suit an imperative style, and lots of keywords would make the code more explicit, given the high stakes.
- Next, turmoil in the nascent Tezos ecosystem lead me to design CameLIGO. Then came ReasonLIGO (by Sander Spies), JsLIGO, and even PyLIGO, which was never plugged in — we did not want to step on SmartPy's feet.

## Architecture

- Nowadays, only two LIGO syntaxes remain: CameLIGO and JsLIGO. The former is almost a strict subset of OCaml, if we leave aside the Tezos-specific literals, a local type definition construct, name punning of record field types, and attributes (they occur in prefix position in LIGO).
- JsLIGO started also as a subset of TypeScript, but evolved and became syntactically incompatible with its model. For example, I implemented a proposal for pattern matching in TypeScript.
- The front-end is made, in order, of a preprocessor, a lexer, two parsers and the first tree transformation before type inference. Each pass can be built as a standalone executable for testing prefixes of the pipeline.
- I wrote the preprocessor because, at the time, we had no time to design and implement a module system, together with a build system.
- Unfortunately, it became very popular, even when LIGO got itself a module system, because it enabled conditional compilation (often for new type definitions of the storage, depending on the version of the standard being implemented).

## Preprocessor and Lexer

- The preprocessor is not trivial, as it needs to understand UTF-8 encoded glyphs (in comments only) and the kind of comments and strings that depend on the LIGO variant.
- The lexer is also a functor parameterised by a lexer that is specific to a given LIGO syntax (comments, strings, keywords etc.). Error reporting has to be aware of linemarkers left by the preprocessor. Some style constraints are enforced (like a linter) for the sake of clarity.
- The lexer is specified as an input to ocamllex (sedlex would perhaps have been preferable).
- Until JsLIGO was added to the party, the lexer signature was a function that returned a single token, which the parser called on demand. Syntactic constructs difficult to parse sometimes rely on so-called *lexer hacks*, that is, having the parser informing the lexer of the syntactic left-context. Instead, I had the lexer run on all the (preprocessed) input, before applying a series of self-passes on the tokens — injecting virtual tokens when needed to help the parser. This is not as expressive as having the syntactic context, but (unbounded) rational approximations work well in practice.

- Perhaps the most interesting self-pass on the tokens is found with JsLIGO:

```
1 :         const f = ([x,y]) ...
```

could be a parenthesised pair or the start of an arrow function (a shift-reduce conflict). Analysing the lexical right-context in the grammar helps disambiguate the two cases, and the lexer injects a ES6FUN token in front of a function:

```
CONST IDENT EQ ES6FUN LPAR ...
```

The parser knows a function is coming next upon reading the virtual token ES6FUN. Unwanted injections can simply be ignored in the grammar.

- A classic one in JsLIGO again is needed to handle ">>" in

```
1 :         type t = u<v<w>>
```

for which a self-pass injects a Zero-Width SPace (ZWSP) virtual token between the two closing chevrons:

```
TYPE IDENT EQ IDENT LCHEV IDENT LCHEV IDENT RCHEV ZWSP RCHEV
```

The parser can then distinguish the binary operator ">>" from the context.

- The LIGO pipeline is more than a compiler, because it provides support for a LSP, and that is why the front-end has to keep all the source information, including comments. This adds significant complexity to the lexer, which has to hide it from the parser. I also wrote source pretty-printers using Pottier's beautiful, backtracking, combinator-based library PPrint (forget Format).
- The two LIGO parsers are generated by Menhir.
- Menhir accepts LR(1) grammars, with semantic actions in OCaml. In practice, many grammars are annotated with precedence declarations to solve LR-conflicts, potentially making the grammars not LR(1), and making it difficult to reason about them.
- Both CameLIGO and JsLIGO are given pure LR(1) grammars, thanks to the rewriting of productions, and the use of virtual tokens.
- Yann Régis-Gianas wrote the front-end of Menhir, and François Pottier the automaton construction and the back-end. The original interest was an application of GADTs to the static typing of the parse stack: `https://cambium.inria.fr/~fpottier/publis/fpottier-regis-gianas-typed-lr.pdf`

# Menhir (example)

This is how the *dangling else* problem is dealt in CameLIGO with Menhir, without resorting to precedence annotations:

```
if_expr(right_expr):
  if_then(right_expr) | if_then_else(right_expr) { ... }

if_then(right_expr):
  "if" expr "then" right_expr { ... }

if_then_else(right_expr):
  "if" expr "then" closed_expr "else" right_expr { ... }

closed_expr:
  no_if_expr(closed_expr)   { ... } (* Defined elsewhere *)
| if_then_else(closed_expr) { ... }
```

- Menhir can identify all error states in the LR-automaton, so we can write precise error messages for them. They are precise because the stack and syntactic right-context is available at the error state, instead of only the next valid tokens, so the past (the intention of the programmer, inferred from the prefix of the stack in terms of grammatical productions) can be expressed in terms of constructs, and the future too, if we take some care when writing the grammar. `https://cambium.inria.fr/~fpottier/publis/bour-pottier-reachability.pdf`
- The fun part of the JsLIGO grammar is the handling of the (in)famous *Automatic Semicolon Insertion (ASI)*. JavaScript has a set of dynamic heuristics, depending on line breaks, allowing in some contexts to omit a semicolon. Instead, I analysed the grammar and hardcoded a sub-grammar (~100 LOC) for statements that allow some semicolons to be missing (and actually can omit more than the mainstream compilers).
- One last very cool paper about using Menhir to parse C11 (as in CompCert): `https://cambium.inria.fr/~fpottier/publis/jourdan-fpottier-2016.pdf`

# Error recovery with Menhir

- Menhir has several interfaces, improperly called APIs, like the *Incremental API* which enables the client to take full control when an error is encountered.
- The parse stack (of terminal and non-terminals recognised so far) and the automaton state (e.g. the LR-items) are available for inspection, as persistent values.
- This enables to decide to either handle the error or try and recover from it: a strategy has to be provided by the client.
- LIGO features error recovery for both CameLIGO and JsLIGO. Several continuations are possible for recovery: skipping some input tokens, popping items from the parse stack, or inserting a synchronisation token. The decision is made based on a heuristic derived from a study of the grammar and common cases (design patterns, keywords and symbol uses, tabulations etc.).
- The LSP makes constant use of this feature to maintain the display of types and documentation on hover, and for go-to-definition on contracts being edited.

- The parser produces an Abstract Syntax Tree (AST) — confusingly named Concrete Syntax Tree (CST) in the code base for legacy reasons: it used to be a parse tree.
- The AST is then transformed into a more abstract kind of tree, the *unified AST*. Whereas the CSTs are specific to each LIGO variant, the unified AST is common to all, and can be considered as the entrance to the common pipeline.
- Because it is the target of a functional and imperative language, respectively CameLIGO and JsLIGO, the definition of the Unified AST is cumbersome. Also, in order to enable some measure of decompilation (type expressions for the LSP), it cannot be too abstract.

# JsLIGO concrete syntax

```
1 : type parameter = unit;
2 : type storage = unit;
3 : type t = [list<operation>, storage];
4 :
5 : @entry
6 : function give5tez (param: parameter, storage: storage): t {
7 :   let operations: list<operation> = [];
8 :   if (Tezos.get_balance() >= 5tez) {
9 :     const receiver =
10 :       match(Tezos.get_contract_opt(Tezos.get_sender())) {
11 :         when(Some(contract)): contract;
12 :         when(None): failwith("Couldn't find account");
13 :       };
14 :     operations =
15 :       [Tezos.Operation.transaction(param, 5tez, receiver)];
16 :   }
17 :   return [operations, storage];
18 : }
```

Even though the syntax error messages displayed by the TypeScript Playground, for example, are relatively good, the JsLIGO parser enables to be correct about the past and complete about the future. For instance:

```
File "syn_err.jsligo", line 2, characters 9-11:
  1 | interface I {
  2 |   type t += int;
  3 | }
Ill-formed type declaration in an interface.
At this point, one of the following is expected:
  * the assignment symbol '=' followed by a type;
  * type parameters (variables) between chevrons ('<', '>') if the
    type is generic;
  * a semicolon ';' if the type is abstract;
  * a closing brace '}' if no more entries.
```

# Approximate size of the front-end

The current front-end is roughly 45 kLOC:

| | |
|---|---|
| Preprocessor library | 3,782 |
| Lexer library | 1,592 |
| Parser library | 1,215 |
| Preprocessor | 349 |
| Lexer | 6,160 |
| Two parsers and pretty-printers | 7,485 |
| Two syntax error specifications | 14,621 |
| Two CSTs | 6,657 |
| Two mappings to unifed AST | 1,609 |
| Sundry libraries | 1,569 |
| TOTAL | 45,039 |

Remember that, at one point, there was also PascaLIGO (13,094 LOC) and ReasonLIGO (data not available) — not counting poor PyLIGO, shell scripts and dune files.

## JsLIGO versus TypeScript

- Despite that the subset of TypeScript needed for writing JsLIGO contracts is comfortable enough (some computations can be expressed in several ways, for instance, and injection of Michelson code is possible), a syntax error cannot tell whether the input is actually valid TypeScript or not.
- For example, in the previous example, the parser cannot actually tell that the input is not valid TypeScript, and the penultimate example is valid JsLIGO but not correct TypeScript.
- Likewise, a type error cannot tell whether the source is valid TypeScript. Worse, since the LIGO type system is *ad hoc*, the inferred types may differ from those of a TypeScript compiler (for example, due to different implicit type coercions).
- The LIGO tooling (LSP, pretty-printers, decompilers etc.) is custom-made.

In short: Idioms and design patterns commonly used by TypeScript developers are not supported in JsLIGO; it is not possible for the LIGO compiler to tell whether it is because the input is not valid TypeScript. If the code compiles, but the semantics differs, the LIGO team would consider it a bug, but it could be silent (we never found an example, though).

## The parser generator tree-sitter

- In order to leverage existing TypeScript tooling, for instance, a pretty-printer,
- to have the LIGO compiler inform the developers of the differences between TypeScript and JsLIGO,
- and to avoid writing a lexer and parser for TypeScript with ocamllex and Menhir, a new approach was proposed, namely use a third-party TypeScript parser.
- Since those are most often closed-source or blatantly incomplete, we decided to use a parser in C generated by tree-sitter from a TypeScript grammar.
- It is difficult to assess the correctness, let alone the conformance, of that parser.
- First, contrary to Menhir, which has a Coq back-end, we do not have a way to prove that the parser indeed recognises the language generated by the grammar, and is complete, that is, exactly the valid inputs are accepted. (Fun/Sad fact: termination is not proved with "`menhir --coq`"!)
- Second, there is no more formal grammar of TypeScript published by Microsoft since version 1.8 (current is 5.8.3) — in 2016.

- tree-sitter has been designed to generate parsers in C from LR(1) grammars.
- Those parsers perform *error recovery*: when encountering an unexpected token, the parser can either *skip* any number of tokens from the input, or *pop* items from the parse stack, or *insert* one synchronisation token when one is deemed missing in the input.
- Instead of letting the client chose one strategy, as Menhir does, the tree-sitter-generated parser runs internally a *Generalised LR* (GLR) algorithm, so, in general, the parse stack is actually a tree of valid prefix sentences (a.k.a. sentential forms). Skipping, popping and inserting create new branches.
- Each branch of the tree maintains two quantities: an *error cost* and a *node count*, respectively, an integer function of the number of skipped tokens, popped items and inserted tokens, and the number of valid subtrees parsed since the last error.
- One heuristic prunes branches ending in an error based on those two values, and another selects the final parse tree if many valid were found (smaller and earlier trees are preferred).

- tree-sitter will always produce a parse tree (CST), but it might contain *error nodes* if skipping or popping was performed during parsing, and *missing nodes* if synchronisation tokens were inserted (one common occurrence is a failure of ASI).
- If a non-terminal was popped from the parse stack, an error node is then shifted instead (pushed) with the subtree of the non-terminal attached below.
- This automatic error recovery mechanism illustrates the fact that tree-sitter was designed, and is mostly used, for tooling, not compilation: as much input as possible must be parsed, no matter how broken.
- Tooling is required to work on syntactically invalid inputs (as it is being edited), whereas compilers mostly care about syntactically valid inputs in batches.
- This difference means that it is often difficult to actually find out if an error was encountered by a tree-sitter-generated parser — let alone how many of them.
- Another difficulty is that errors are nodes in the parse tree, so it can be rather difficult to express them, and their remediation, in terms of the grammar.

For example, given a file named `broken.ts`:

```
1 :       type t += int;
```

if we call "`tree-sitter parse broken.ts`", we get:

```
(program [0, 0] - [1, 0]
  (type_alias_declaration [0, 0] - [0, 14]
    name: (type_identifier [0, 5] - [0, 6])
    (ERROR [0, 7] - [0, 8])
    value: (type_identifier [0, 10] - [0, 13])))
broken.ts Parse:    2.41 ms     6 bytes/ms (ERROR [0, 7] - [0, 8])
```

While this is one of the simplest kinds of error, we can already see that an ERROR node can be inserted *anywhere*, with respect to the grammar, which looks like this:

```
type_alias_declaration: $ => seq(
  'type',
  field('name', $._type_identifier),
  field('type_parameters', optional($.type_parameters)),
  '=',
  field('value', $.type),
  $._semicolon)
```

Another remark is that the parse tree is not exactly what a compiler designer would call it — and need, because it does not contain keywords, symbols or identifiers. The rationale is, as we mentioned above, that tree-sitter is meant to be used in conjunction with a text editor, where the input is already in memory, so retrieving lexemes (to wit, concrete representation of tokens) is efficient.

- As seen above, a tree-sitter grammar is a JavaScript declaration.
- In the case of involved syntaxes, it is often the case that writers of LR grammars resort to annotations to resolve LR-conflicts (shift/reduce or reduce/reduce).
- This is not what I had done with CameLIGO and JsLIGO in order to guarantee the LR(1) property, but this is what was done for the tree-sitter grammars of JavaScript and TypeScript: about 45-50 precedence annotations in total.

```
// Oh boy
// The issue is these special type queries need a lower relative precedence
// than the normal ones, since these are used in type annotations whereas the
// other ones are used where `typeof` is required beforehand. This allows for
// parsing of annotations such as
// foo: import('x').y.z;
// but was a nightmare to get working.
```

- What is worse is that `MISSING` nodes are actually not named nodes, like `ERROR` nodes are. Instead, they are metadata to the closest node where the insertion of a synchronisation token occurred, and that token has length zero.
- From the point of view of an OCaml developer, interfacing with a tree-sitter-generated parser is a source of concern because it provides a parse tree in C with a great number of kinds of node, and it is naturally accessed through pointer arithmetics.
- The first thing we did (Leo Unoki and Alistair O'Brien) was to use the `ocaml-ctypes` Foreign Function Interface (FFI).
- I highly recommend that you read the amazing paper by Jeremy Yallop and others about its design and implementation: `https://www.cl.cam.ac.uk/~jdy22/papers/a-modular-foreign-function-interface.pdf`

- The general idea is to provide a type-safe and modular interfacing to C code, where both C types and C functions correspond to OCaml values built from the `ocaml-ctypes` library.
- It is modular in the sense that using OCaml functors, we can switch from dynamic linking to static linking, for instance. In the case of the new JsLIGO, we use the latter.
- The most difficult part is actually building the piece of code that uses the library: the dune file is challenging because of the generation of OCaml code.
- For the rest, two files are needed: one for interfacing the types, and one for the functions.

# Interfacing types

Here is how to define an OCaml type corresponding to a C type:

```
1 :  open Ctypes
2 :
3 :  (* Single source location:
4 :
5 :     typedef struct TSPoint {
6 :       uint32_t row;
7 :       uint32_t column;
8 :     } TSPoint;
9 :  *)
10 :
11 :  type ts_point
12 :
13 :  let ts_point
          : ts_point structure typ = structure "TSPoint"
14 :  let row      : _ field = field ts_point "row" uint
15 :  let column   : _ field = field ts_point "column" uint
16 :  let () = seal ts_point
```

# Interfacing functions

Here is how to define an OCaml function calling a C function:

```
(* Call a parser on a contiguous buffer.

   TSTree *ts_parser_parse_string(
     TSParser *self,
     const TSTree *old_tree,
     const char *string,
     uint32_t length
   );
*)

let ts_parser_parse_string =
  foreign
    "ts_parser_parse_string"
    (ptr ts_parser @-> ptr ts_tree @->
          string @-> uint32_t @-> returning @@ ptr ts_tree)
```

# Example of parse of TypeScript

```
1 : (* Parsing some TypeScript code as a string *)
2 :
3 : let parse_typescript_string source_code : ts_tree_ptr =
4 :   let parser = TS_fun.ts_parser_new ()
5 :   and language = tree_sitter_typescript () in
6 :   let _ = TS_fun.ts_parser_set_language parser language in
7 :   let null_tree = from_voidp TS_types.ts_tree null in
8 :   let parse_tree =
9 :     TS_fun.ts_parser_parse_string
10 :       parser
11 :       null_tree
12 :       source_code
13 :       (UInt32.of_int @@ String.length source_code)
14 :   in
15 :   TS_fun.ts_parser_delete parser;
16 :   parse_tree
```

# Example of parsing TypeScript

```
1 :  let code = "[1, null]" in
2 :  let tree : ts_tree_ptr = parse_typescript_string code in
3 :  let program_node
           : ts_tree = TS_fun.ts_tree_root_node tree in
4 :  let of_int = UInt32.of_int in
5 :  let expr_stmt_node : ts_tree =
6 :    TS_fun.ts_node_named_child program_node (of_int 0) in
7 :  let array_node : ts_tree =
8 :    TS_fun.ts_node_named_child expr_stmt_node (of_int 0) in
9 :  let number_node : ts_tree =
10 :   TS_fun.ts_node_named_child array_node (of_int 0) in
11 :  let null_node : ts_tree =
12 :   TS_fun.ts_node_named_child array_node (of_int 1) in
13 :  (* Checking the node types *)
14 :  assert (string_of_ts_node_type program_node = "program");
15 :  assert (string_of_ts_node_type
           expr_stmt_node = "expression_statement");
16 :  assert (string_of_ts_node_type array_node = "array");
17 :  assert (string_of_ts_node_type number_node = "number");
18 :  assert (string_of_ts_node_type null_node = "null");
```

# Abstracting the FFI

In order to prevent null pointers to be dereferenced through the API, I created a layer on top of the library above, returning either optional or result values.

```
let collect ?(comments = false) select_child arity node =
  if is_null node then []
  else
    let rec fold acc n =
      if UInt32.(equal zero n) then acc
      else
        let index = UInt32.pred n in
        let child = select_child node index in
        match get_name child with
        | "comment" when not comments -> fold acc index
        | _ -> fold (child :: acc) index
    in fold [] (arity node)

let collect_named_children
     ?(comments = false) (node : ts_tree) : ts_forest =
  TS_fun.(collect ~comments
       ts_node_named_child ts_node_named_child_count node)
```

# Printing the CST

- The next step was to write a printer for the CST, so that null pointers, empty nodes (that is, with empty range of source locations) and ERROR nodes are printed, so they can be retrieved easily by a regular expression.
- A crucial issue is that, on the one hand, we have the TypeScript grammar (we saw above an example), and, on the other hand, the CST queried through the foreign interface is one particular parse tree, but we need to be able to decode *all possible parse trees*.
- In other words, we find ourselves in the awkward situation of having to write by hand a parser for each grammar rule!
- This is because normal usage of tree-sitter is not for compilation, but for reading syntactically invalid inputs — not build another tree.
- Another source of concern is the retrieval of as many comments as possible, not just for the (future) LSP, but also because we will need to collect from them decorators, in places where the TypeScript syntax does not allow them.

# Printing the CST

I reused a tiny library I wrote to debug the current CSTs. For example:

```
1 :       type t += int;
```

```
program (broken.jsligo:1:0-2:0)
`- type_alias_declaration (broken.jsligo:1:0-14)
   |- ERROR (broken.jsligo:1:7-8)
   |  `- +
   |- type [keyword] (broken.jsligo:1:0-4)
   |- type_identifier (broken.jsligo:1:5-6)
   |  `- t
   |- = (broken.jsligo:1:8-9)
   `- type_identifier (broken.jsligo:1:10-13)
      `- int
```

Note how I recovered the lexemes. (Compare with what tree-sitter printed above.)

# Printing the CST

```
1 : and print_type_alias_declaration ?(comments = []) state node =
2 :   match get_name node with
3 :   | "ERROR" | "MISSING" | "NULL" ->
4 :     print_error_node state node ~err:Type_alias_declaration
5 :   | _ ->
6 :     let comments = comments @ prev_comments node
7 :     and kwd_type = first_child_named "type" node ~err:Type
8 :     and name_field = child_with_field "name" node ~err:Type_name
9 :     and sym_equal = first_child_named "=" node ~err:Equal
10 :    and type_parameters_field = child_with_field_opt "type_parameters" node
11 :    and value_field = child_with_field "value" node ~err:Type_expression in
12 :    let children =
13 :      [ mk_child_res (mk_kwd_type ~comments) kwd_type
14 :      ; mk_child_res print_identifier name_field
15 :      ; mk_child_opt print_type_parameters type_parameters_field
16 :      ; mk_child_res mk_sym_equal sym_equal
17 :      ; mk_child_res print_type value_field
18 :      ] in
19 :    let error_child = Ts_wrap.first_child_named_opt "ERROR" node in
20 :    let children = mk_child_opt make_node error_child :: children in
21 :    make_tree state node children
```

- Lines 3-4 handle the error cases. Line 6 gathers the comments inherited and before the non-terminal we are parsing. Lines 7-11 query (parse) the children node.
- Lines 12-21 build the corresponding tree, and commit it to the current state. Note lines 19-20 that insert a single ERROR node amongst the children if at least one is found. (See example above.)
- The bare-bones CST printer is 4,268 LOC. It is not callable from the pipeline, only as a standalone binary.
- I ran the printer on 5,609 TypeScript files from the Microsoft conformance test suite, and looked for ERROR nodes (coming from the C CST), error nodes MISSING and NULL that the abstraction layer (on top of the foreign function interface) might have created. If NULL nodes are present, then a null pointer was intended to be dereferenced and that would point to a wrong interpretation of the grammar.
- Everything was fine. It was time consuming because the test suite mixes positive and negative tests, but, at least, we made sure that no null pointer showed up.

# Decoding the CST

- After testing the CST printer, I moved to write a decoder that would query the CST as the printer does, but, instead of printing it, an AST as an OCaml value would be built.
- It systematically makes use of the result type to handle errors, with support from monadic let syntax (`let*`).
- The design of the AST was error-prone because of the semantics of tree-sitter grammars: such grammars can reuse others which either augment a given production with more cases, shadow a production or add new productions.
- In this instance, I had to read the JavaScript and TypeScript grammars like synoptic gospels.

# Decoding the CST

```
1 : and dec_type_alias_declaration
          ?(comments = []) node : (type_alias_declaration, _) result =
2 :   match get_name node with
3 :   | "ERROR" | "MISSING" | "NULL" -> mk_err Type_alias_declaration node
4 :   | _ ->
5 :     let comments = comments @ prev_comments node in
6 :     let* kwd_type = first_child_named "type" node ~err:Type in
7 :     let* kwd_type = dec_kwd_type ~comments kwd_type in
8 :     let* name_field = child_with_field "name" node ~err:Type_name in
9 :     let name = dec_type_identifier name_field in
10 :    let* sym_equal = first_child_named "=" node ~err:Equal in
11 :    let* sym_equal = dec_sym_equal sym_equal in
12 :    let type_parameters_field
              = child_with_field_opt "type_parameters" node in
13 :    let* type_parameters
              = make_opt_res dec_type_parameters type_parameters_field in
14 :    let* value_field
              = child_with_field "value" node ~err:Type_expression in
15 :    let* type_expr = dec_type value_field in
16 :    let* () = check_first_error_child node in
17 :    Ok { kwd_type; name; type_parameters; sym_equal; type_expr }
```

- Lines 3-4 handle the error cases. Line 5 gathers the comments inherited and before the non-terminal we are parsing.
- Queries against the CST and decoding are interleaved. For instance, line 6 queries the keyword "`type`", and line 7 decodes it, that is, makes a value for the AST node under construction (see line 17 that assembles it in the absence of errors).
- Line 16 looks for the presence of an ERROR child, and fails with a generic "Syntax error" message, whereas other failures at decoding carry more useful messages: those are denoted by the named arguments `~err:...`.
- The bare-bones CST decoder is 8,903 LOC (including the AST).

# Decoding errors

Let us decode (with a standalone executable, like the CST printer) the same broken piece of code as printed above:

```
1 :       type t += int;
```

```
File "Tests/broken.jsligo", line 1, characters 7-8:
  1 | type t += int;
Error: Syntax error.
```

# Decoding errors

Another example, with a better error message (but not as awesome as what Menhir can do for us):

```
1 : interface I {
2 :   type t
3 : }
```

```
File "Tests/broken.jsligo", line 2, characters 7-8:
  1 | interface I {
  2 |   type t
  3 | }
Error: A field for the object type is expected.
```

("type" is a so-called *soft keyword*, that is, in certain contexts it is interpreted as a keyword, in others an identifier.)

# Stripping the AST

The pass after decoding consists in stripping the AST to a proper JsLIGO AST, which we called *stripped AST*. For instance, we go from

```
and type_alias_declaration = {
  kwd_type : kwd_type;
  name : type_identifier;
  type_parameters : type_parameters option;
  sym_equal : sym_equal;
  type_expr : type_expr}
```

to

```
and type_decl = {
  name : variable;
  generics : variable list;
  type_expr : type_expr}
```

# Stripping the AST

The code that performs the transformation above is:

```
1 : and strip_D_type_alias_declaration
        (node : Ast.type_alias_declaration wrap)
2 :     : (S.declaration, _) result
3 :   =
4 :   let type_decl, region = node#payload, node#region in
5 :   let Ast.{ kwd_type = _; name; type_parameters
          ; sym_equal = _; type_expr } = type_decl in
6 :   let name = strip_type_identifier name in
7 :   let* generics = strip_list_opt
          strip_type_parameters type_parameters in
8 :   let* type_expr = strip_type_expr type_expr in
9 :   let type_decl = S.{ name; generics; type_expr } in
10 :  Ok (S.D_type (mk_reg region type_decl))
```

The AST stripper is 3,606 LOC (including the stripped AST).

The AST stripper enables to reject some syntactically valid TypeScript programs that are not accepted by JsLIGO. For example,

```
1 : type t = int[];
```

```
File "Tests/broken.jsligo", line 1, characters 9-14:
  1 | type t = int[];
Error: The type 'array' is not supported by JsLIGO.
```

This is not possible with the Menhir-generated parser because its grammar does not describe all of TypeScript.

There is a new tool and two new passes:

1. the CST printer, for getting the tree traversal and queries right;
2. the CST decoder, which queries the C CST through the abstraction of the FFI, and builds the TypeScript AST;
3. the AST stripper, which filters down the TypeScript AST to the valid syntactic constructs of JsLIGO.

An existing pass had to be entirely rewritten: the transformation from the stripped AST to the unified AST (798 LOC).

The total size of the JsLIGO front-end is 21,642 LOC (including the new mapping to the unified AST and the printer).

It can be argued that the next pass, after the unified AST, is also part of the front-end, because the middle-end starts with the type-checker, but it is probably off-topic here.

# Syntactic changes of JsLIGO

We lose the Tezos-specific literals:

```
const zero = 0x;
const one = 1n;
```

is now

```
const zero = "" as bytes;
const one = 1 as nat;
```

Parameters of contracts change from

```
parameter_of A.B.C
```

to

```
parameter_of<A.B.C>
```

# Syntactic changes of JsLIGO

No more field name punning in types and expressions:

```
const a = { x };
```

is now

```
const a = { x: x };
```

Separators are not so versatile with the TypeScript grammar we use:

```
{ x: 4; y }
```

must use commas now:

```
{ x: 4, y }
```

We kept variant types as a special interpretation of union types: when all the summands are tuples whose first component is a singleton string type. Expressions constructed for those types expect a type coercion to their original singleton type.

```
type option<T> = ["Some", T] | ["None"];
const some_one = Some(1);
```

is now

```
type option<T> = ["Some", T] | ["None"]; // Same as before
const some_one = ["Some" as "Some", 1];
```

If there is only one variant, it must start with a vertical bar:

```
type singleton = | ["Single"]
```

# Syntactic changes of JsLIGO

We mentioned above that I added a proposal for doing pattern matching in TypeScript. Since that proposal has not been adopted yet, we must find another way. We used to have:

```
1 :     const is_some =
2 :       match(some_one) {
3 :         when (Some(_)): true;
4 :         when (None()): false
5 :       }
```

Now, we assume a predefined function `$match` and the cases become object properties:

```
1 :     const is_some =
2 :       $match(some_one, {
3 :         "Some": n => true,
4 :         "None": () => false
5 :       })
```

Because this new syntax expects names for data constructors, like `Some`, we cannot use `[]` as a pattern matching the empty list. I simply added to the LIGO standard library a function `List.head_and_tail`:

```
function rev <T>(xs : list<T>) : list<T> {
  const rev = <T>([xs,acc]: [list<T>, list<T>]) : list<T> =>
    $match(List.head_and_tail(xs), {
      "None": () => acc,
      "Some": ([y,ys]) => rev([ys, [y,...acc]])
    });

  return rev([xs, []]);
};
```

A big change is the unplugging of the preprocessor and the development of a new build system (by Dmitry Laptov) that enables three forms of import statements to replace `#import`:

```
import M = M.O;
```

remains the same, but now we can use

```
import * as M from "/my/path.jsligo"
```

and

```
import {x, y} from "/my/path.jsligo"
```

Note that this does not replace existing uses of `#include` and `#if` for conditional compilation and as a guard against double inclusions: manual upgrading is needed.

In TypeScript, valid locations for decorators is rather limited in comparison with JsLIGO. As a consequence, we have sometimes to put them in comments just before the construct they apply to. For example, at the top-level,

```
@entry
const add = (s: int, k: int) : [list<operation>, int] => [[], s+k];
```

becomes

```
// @entry
const add = (s: int, k: int) : [list<operation>, int] => [[], s+k];
```

# Syntactic changes of JsLIGO

Interfaces in JsLIGO could contain type definitions, including of abstract types, and entries could be decorated. This is no more valid.

```
interface FA0 {
  type storage = int;

  @entry
  const add: (p: int, s: storage) => [list<operation>, storage];
}
```

is now

```
type storage = int;

interface FA0 {
  // @entry
  add: (p: int, s: storage) => [list<operation>, storage];
}
```

Sadly, there is no way for now to recover the loss of type abstraction.

Namespaces were able to implement interfaces, which is not possible in TypeScript. Instead we use now a class:

```
namespace Impl implements FA0 {
  @entry
  const add = (p: int, s: int) : [list<operation>, int] => [[],p+s];
}
```

is now

```
class Impl implements FA0 {
  @entry
  add = (p: int, s: int) : [list<operation>, int] => [[], p+s];
}
```

I implemented do-expressions in JsLIGO for the right-hand sides of pattern matchings, which are expected to be expressions: sometimes we need to evaluate some statements before returning a value. Before

```
do { ... return e }
```

and now, we use a thunk instead:

```
(() => { ... return e })()
```

If some variable v cannot be captured in the thunk above, because of Michelson constraints, we simply pass it as an argument, like so:

```
((v) => { ... return e })(v)
```

# Conclusion

- The test contracts and the code excerpts in the LIGO documentation have been upgraded. All meaningful tests pass (I disabled various kinds of LSP tests and pretty-printing tests, for example).
- The documentation in English is being updated (Tim McMackin).
- Since we are breaking things, it is time to discard deprecated modules from the LIGO standard libray, e.g. replace the contents of `Tezos` by that of `Tezos.Next`.
- Since the LIGO registry is being deprecated, some standard contracts in JsLIGO will have to be distributed in some other way and updated manually.
- Tooling is not available yet (LSP): this is a non-trivial undertaking.
- An experimental release will be done without the tooling.
- CameLIGO will still work as usual, but the plan is to release it separately later as a library for OCaml developers (Eduardo Rafael), instead as a standalone compiler.
- For your information, the back-end now uses LLTZ, that is, benefits from SmartPy's optimisations (Alan Marko).